BusBidder Spain: Personalized Bus Journey Comparison and Booking System

Kyle Strother

November 3, 2026

Contents

1 Introduction **2**

2 Requirements and Problem Interpretation **2**

3 System Architecture and Software Design **3**

3.1 Data Flow Diagram 3

3.1.1 Trip Data Ingestion and Storage 4

3.1.2 Priority Evaluation and Ranking System 4

3.1.3 User Management and Graphical Interface 4

3.2 Object-Oriented Design Decisions 4

4 Data Management and Validation **5**

5 Searching, Filtering, and Scoring Methodology **6**

5.1 Searching and Filtering Trip data 6

5.2 Weighted Ranking Methodology 6

5.2.1 Code implementation 8

6 Web Interface and Booking Functionality **9**

7 Testing and Experimental Results **10**

8 Traveller Profile Comparisons **12**

9 Critical Evaluation and Limitations **20**

10 AI Declaration of Use **22**

1 Introduction

BusBidder is a bus-booking marketplace that allows users to compare offers from five bus companies. It tries to simplify journey selection by filtering unsuitable offers and ranking the remaining options based on user's priorities.

Comparing bus offers is difficult because the cheapest ticket isn't necessarily the best choice. Differences in duration, departure times, transfers, and flexibility can affect an offer's value to a user. Few users agree on what matters most for a trip.

BusBidder's objective is to provide personalised filtering and a weighted ranking system, that'll help travellers identify the most suitable journey based on their requirements and priorities.

2 Requirements and Problem Interpretation

The application achieves this by:

- loading and validating Trip data
- removing Trips that don't meet hard constraints set by users
- scoring the remaining offers using a weighted ranking model set forth by individual users
- presenting explainable recommendation
- supporting simulated bookings for multiple users

The application uses a modular, object-oriented Python design to separate functional responsibilities such as data management, validation, filtering, scoring, and booking. A weighted scoring model allows different criteria to be normalised and combined into a measurable score. This scoring and its composition is created in a way that isaccessible and can be referenced outside of internal objects in a way that allows explanation of ranking to users.

A consideration in this applications design is Hard versus Sorft Constraints. Hard constraints determine whether an offer is acceptable and can be set as a hard constraint by the user on any field given such as destination, travel dates, budget and seat availability. Preferences, such as lower prices, shorter journeys, convenient departure times and ticket flexibility, can be Soft constraints left to the user to decide if the influence the ranking but don't automatically exclude an offer from being listed.

3 System Architecture and Software Design

3.1 Data Flow Diagram

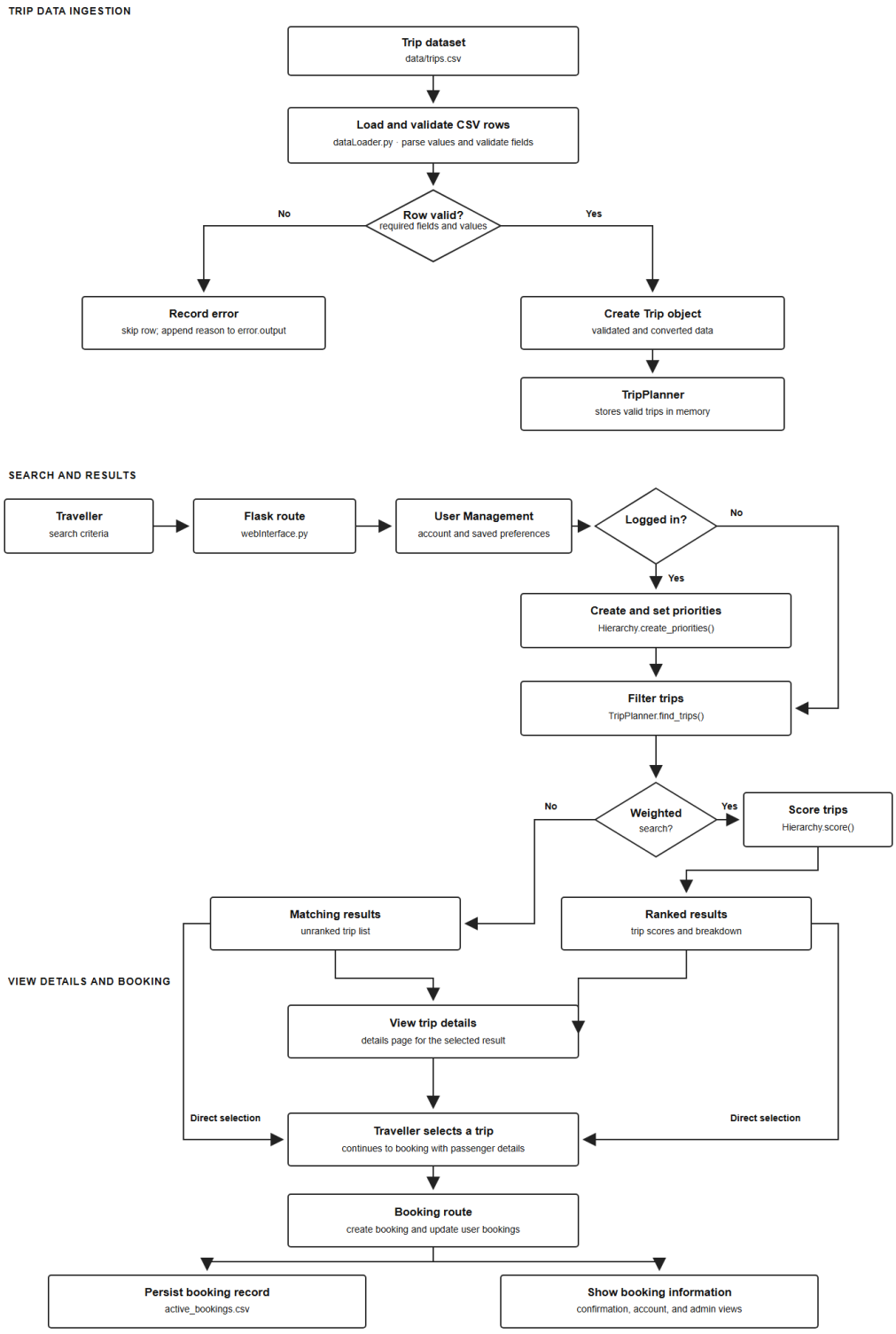


Figure 1: BusBidder data flow from trip ingestion through ranking and booking.

3.1.1 Trip Data Ingestion and Storage

The file `dataLoader.py` loads trip data from the CSV file `data/trips.csv`. The `dataLoader.py` validates it row by row rejecting and reporting failed imports along with reasoning for rejection before they are converted into `Trip` objects. The file `tripsService.py` then manages the storage of `Trip` objects, and then implements the system for querying stored trips, and is also responsible for exporting booking information to external files.

3.1.2 Priority Evaluation and Ranking System

The module `priority.py` determines trip scores and defines priority types, qualifiers, and evaluation methods. The module `priorityHierarchy.py` manages priorities and their weights and initiates scoring. The file `scoreObject.py` stores trip scores and priority breakdowns.

3.1.3 User Management and Graphical Interface

The class definitions in `user.py` handle the user account data and model, stores a users weighted priorities; and also establishes a definition of a `Booking` object. The `webInterface.py` module defines and runs the Flask application and exports failed data imports and reasoning to the external file `error.output`. It's responsible for exposing the application routes and page rendering logic. It also handles the user input, validating it and querying the data against it. It's through this module that the user can create an account, log in, and save their priority hierarchy, and search for trips have the system display a ranked list of results. The user can then select a trip to book, and the system will save the fare conditions associated with the booking information and tie it to a unique booking reference utilizing the an incrementor and the offerID while updating the Trip data to reduce the number of available seats/capacity.

Due to the decision to not include data base logic, the module also handles the persistence of user data and bookings to external files.

3.2 Object-Oriented Design Decisions

The use of separate modules in this design allows the ranking system to operate independently from the web interface, which allowed for better organization and maintainability. Most OOP design decisions concern the priority evaluation and ranking system, the application's core. Inheritance, composition, and dynamic behaviour are relevant to the project. A user can maintain a priority hierarchy that changes at runtime and isn't fixed once created. Designing priorities in a user specific hierarchy allows simultaneous users to hold their own preferences.

In designing the priority system, implementing an abstract `EvaluationMethod` to be overwritten by the `Binary` Evaluation class and the `Comparative` Evaluations class within the priority object allows Trip scoring to adapt to different priority types without changing the application model.

4 Data Management and Validation

The dataset for this program is stored in a CSV file, with each row representing a bus trip and its associated attributes. The columns include the following information:

- `offer_id` Required
- `company` Required
- `destination` Required
- `travel_date` Required
- `arrival_date` Required
- `departure_time` Required
- `arrival_time` Required
- `duration_minutes` Required
- `price_euro` Required
- `transfers` Required
- `transfer_via` Optional
- `luggage_allowance_kg` Required
- `changeable` Required
- `refundable` Required
- `available_seats` Required
- `customer_rating` Required
- `rating_scale_max` Required
- `environmental_measure_kg_co2e_per_passenger` Optional

The contents of the supplied dataset are a collection of bus trip offers from various companies, accross different destinations and dates. Information on 5 sample companies and 15 valid destinations allowed for the generation of this data.

Each CSV row is checked field by field: Ensures required ID exists, a valid company and destination are given based on the 5 given companies and 15 given destinations, sensible and valid dates and times are given for departure and arrival, valid numeric values and ranges for fields like price, trip duration,number of transfers, luggage allowance, etc.., validates boolean fields are true or false, and a rating that fits its scale. Optional environmental-impact data is parsed when present. If the checks pass, the function converts the values to suitable Python types and creates a Trip object. If any validation fails, the row is rejected and the reason for rejection is logged to an external file `error.output`.

In the provided dataset, there are a total of 125 Trips with 8 Trips being invalid. The 8 invalid trips were rejected due to the following reasons:

- Invalid price data. Price can not be a negative value.
- Invalid trip destination.
- Invalid date given with a month of 13, which isn't a valid month.
- Arrival Date occurs before the departure date, which isn't a valid trip.
- Invalid customer rating data.
- There needs to be at least 1 available seat for a trip to be valid.
- Price is a mandatory field and can't be empty.
- A negative number of transfers is impossible for a valid trip

5 Searching, Filtering, and Scoring Methodology

5.1 Searching and Filtering Trip data

The `tripsService.py` module is responsible for searching and filtering the stored trip data. It provides methods to query trips based on user-defined criteria, such as destination, travel date, departure time, and other attributes. But none of the parameters are required, allowing queries to range from general to specific. If no options are given, it'll return all trips in the dataset. The User is able to make these queries through the web interface, where they can select their desired criteria and submit the search. Any specified filters are applied to the trip search and any trip that doesn't meet the criteria is excluded from the results.

It's in this Search form, that we create "hard constraints" as any trip that doesn't meet a user's supplied constraints shouldn't be displayed to the user. A search for 11-04-2027 to 11-08-2027 shouldn't show a trip on the 15th. For the implementation of a "soft constraint" we use the weighted ranking system to score and rank trips based on a user's preferences, allowing them to still see non optimal results but display the best options first.

The system setup with allowing a User to have their priorities saved to their account while also allowing them to make rigid searches with hard constraints allows for a flexible and personalized experience. Say a user has a priority set for Lower prices but also knows they can't spend more than 45 Euros, The web form implementation allows them to prevent any trips costing over 45 from being displayed while also displaying trips in a price minded order as well.

5.2 Weighted Ranking Methodology

When a user has a certain idea of the priorities the focus on in a trip are for example Price, Flexibility, Low transfers, short duration, Departure time after 17:00. We can establish a structure that has 100 points and to allow them to rank each priorities importance in respect to a percentage of points. in the following weight:

- Price 37
- Flexibility 26
- Low transfers 19
- Short duration 11
- Departure time after 17:00 7

For this example we can establish 5 test trips as follows:

Company	Destination	Departure time	Duration (min)	Price (EUR)	Transfers	Changeable	Refundable
Camino Express	Barcelona	15:15	399	89.64	0	FALSE	FALSE
CostaLink	Barcelona	13:30	484	60.25	0	FALSE	TRUE
VerdeBus	Barcelona	11:00	513	63.56	0	FALSE	FALSE
NocheVia	Barcelona	20:15	617	53.40	1	TRUE	FALSE
NocheVia	Barcelona	21:15	623	57.22	1	FALSE	TRUE

Table 1: Example bus journeys to Barcelona.

With this hierarchy we see two things: some are measured binarily, and others need a comparison to determine their quality (i.e. what's a lower price in this context).

If we determine minimum and maximum price, and duration of the trips given in an applicable list we can utilize the min max normalization formula to determine the percent of its weighted points a priority is awarded The trip with the lowest price should be given 100the same can be said for the duration and number of transfers

Binary priorities award full points when criteria are met, such as a departure after a specified time.

Flexibility is more complicated, but we can categorize it as 60

With this scale and Trip data, we have:

- max price of 89.64 and a min of 53.40

- trip duration max: 623 and min: 399

- transfers max: 1 and min: 0

With these scales and trip data, the design decision is to use the min max normalization formula

$$X_{norm} = \frac{X - X_{min}}{X_{max} - X_{min}}$$

to determine relative value. The problem here is it isn't universal: A low Price is good, but a low customer rating isn't. For fields where lower values are better we can take the 1 - the normalization to get an appropriate scale.

Trip 1: - Price:

$$X_{price} = 1 - \left(\frac{89.64 - 53.40}{89.64 - 53.40}\right) = 0$$

- Flexibility: Trip has no refunds or changability so

$$0.6\,(0) + 0.4\,(0) = 0$$

- Transfers: No transfers

$$X_{transfers} = 1 - \left(\frac{0 - 0}{1 - 0}\right) = 1$$

Transfers: 19 points

- Duration: This has the shortest duration

$$X_{duration} = 1 - \left(\frac{399 - 399}{623 - 399}\right) = 1$$

11 points

- Departure time after 17:00: this trip departs at 15:15 so it doesn't meet this expectation: 0 points

Trip 1: point total = 30 / 100 Meaning this trip isn't a great fit.

Doing this for all 5 trips we get the following results:

Company	Price	Flexibility	Transfers	Duration	Departure Time	Total Score
Camino Express	0.00	0.00	19.00	11.00	0.00	30.00
CostaLink	30.01	15.60	19.00	6.83	0.00	71.44
VerdeBus	26.62	0.00	19.00	5.40	0.00	51.02
NocheVia	37.00	10.40	0.00	0.29	7.00	54.69
NocheVia	33.10	15.60	0.00	0.00	7.00	55.70

Table 2: Example bus journeys to Barcelona with their respective scores.

We now have a way to score and rank trips based on a user's weighted priorities. The benefit of allowing the user to assign a customizable weight is that even if another user ranks those priorities in the same order, different distributions produce different scores.

5.2.1 Code implementation

Utilizing the Web form in the `webInterface.py` module, the user can select their priorities and assign weights to them, in a graphical interface portrayed as a pie chart. One the user submits their preferences, the system will validate the input and create a `PriorityHierarchy` object with the user's priorities and weights. Deeper within the `priority.py` module, each priority is created with a relevant weight and in cases where it's needed a `Qualifer` object is created to add an operator and a value to compare a trips data against to give its scoring a bit more context for what a user values. A good example of the qualifier is the departure time after 17:00 priority, which has a qualifier of operator of "Later than" and value 17:00. so it knows a custom value to score a trip against. Each `Priority` Object has an attribute `PriorityType` which maps it to a given Trip attribute and designates a specific evaluation method. The PriorityType is an Enum that's used to determine which evaluation method to use for a given priority, there are two evaluation methods, Binary and Comparative, and that's used to determine the scoring metric to score Trips.

`Comparative` evaluates attributes such as price relative to the other matching trips, while `Binary` evaluates conditions such as whether a departure meets a specified time preference.

6 Web Interface and Booking Functionality

This application design has opted to utilize Flask and Jinja for a web interface for user interaction. the Flask Application structure uses python Decorator functions such as `@app.route("/booking", methods=["GET", "POST"])` to route the user to pages allowing a user to customize the flow through functionality and improves the experience by allowing them to search through the Trip data even when not logged in. The application has several routes for the user to navigate:

- An index page that allows navigation to most other pages.
- A login/registration page that lets a user register an account or log in to an existing one. Once valid account data is entered, the user is redirected to a profile management page. New accounts are saved on the server and in `registered_users.csv`.
- A profile management page that allows a user to change account information and site theme, set and weigh priorities, and view details of booked trips.
- A quick search page, available from the index page after login, where the user can specify ticket quantity, destination, and date or date range. The application uses the user's priorities to show up to six matching trips.
- A search page that allows any user to find trips using the search fields. Logged-in users can also rank the results. From the results, users can open a trip details page or go directly to booking.
- A details page that displays information about a trip. For logged-in users, it also displays the trip's scoring breakdown and explains its score.
- A booking page that displays trip information for confirmation. The user can select a number of seats up to the available capacity, enter passenger names, and submit the booking. Bookings are saved on the server and exported to `active_bookings.csv`.

These pages are HTML Flask pages using Jinja Scriptlets to easily acess the python objects, and are styled and given JavaScript interactivity through external scripts. All of these pages exist in the `./templates` directory, and their styling and JavaScript files are in `./static` directory.

7 Testing and Experimental Results

The testing for this application uses Python's built in unittest modules.

the `testing/data_loading/test_dataLoader_unittest.py` checks valid and invalid files, empty files, dates before today, and whether invalid rows produce errors. That folder also contains testing data that intentionally is formatted both correctly and incorrectly, and utilizes the error validation in the `dataLoader.py`'s data ingestion to catch all invalid rows to cover invalid, boundary, and missing data-cases.

`testing/trip_scoring/test_trip_scoring_unittest.py` checks priority weight limits, duplicate priorities, missing time qualifiers, and that scores and breakdowns are produced. Theses tests validate this application's scoring metrics and also verify that our weight-based scoring can systemically organize trip data into relevant to less relevant trip data on a per user basis. The final tests in this module validate that two different user with different priorities rank the test data in a different order.

Console output for the entire testing suite is included in the Appendix.

```
Starting testing.trip_scoring.test_trip_scoring_unittest.TestTripScoring.test_user_based_priority_ranking
Loaded 2 valid test trips.
Comparing rankings for budget-minded and duration-minded users.
Budget-minded ranking: OF001 (70.00), OF002 (60.00)
Duration-minded ranking: OF002 (90.00), OF001 (50.00)
Top trips differ: budget=OF001, duration=OF002.
```

Figure 2: Console output showing different top-ranked trips for two traveller priority profiles.

```
class TestTripScoring(unittest.TestCase):
    def setUp(self):
        print(f"\nStarting {self.id()}", flush=True)
        # Load trips from a test CSV file
        self.trips, errors = load_trips('testing/data_loading/valid_test.csv')
        self.assertEqual(len(errors), 0, "There should be no errors for valid data")
        print(f"Loaded {len(self.trips)} valid test trips.", flush=True)

        # Create a user and set up a hierarchy with priorities
        self.user = User(username="testuser", name="Test User", password="password", email="testuser@example.com")
        hierarchy = Hierarchy()
        hierarchy.add_priority(Priority(PriorityType.LOWER_PRICE, 30))
        hierarchy.add_priority(Priority(PriorityType.DEPARTURE_BEFORE, 20, qualifier=Qualifier(QualifierOperator.EARLIER_THAN, datetime.time(12, 0))))
        hierarchy.add_priority(Priority(PriorityType.ARRIVAL_BEFORE, 20, qualifier=Qualifier(QualifierOperator.EARLIER_THAN, datetime.time(18, 0))))
        hierarchy.add_priority(Priority(PriorityType.DURATION, 10))
        hierarchy.add_priority(Priority(PriorityType.FEWER_TRANSFERS, 10))
        self.user.hierarchy = hierarchy
```

Figure 3: Unit test code showing the budget minded users priority weighting.

```python
class TestTripScoring(unittest.TestCase):

    def test_user_based_priority_ranking(self):
        print("Comparing rankings for budget-minded and duration-minded users.", flush=True)
        planner = TripPlanner()
        planner.add_trips(self.trips)

        short_trip_user = User(username="PreferShort", name="ShortUser", password="password", email="testshortuser@example.com")
        hierarchy = Hierarchy()
        hierarchy.add_priority(Priority(PriorityType.DURATION, 40))
        hierarchy.add_priority(Priority(PriorityType.LOWER_PRICE, 10))
        hierarchy.add_priority(Priority(PriorityType.DEPARTURE_BEFORE, 20, qualifier=Qualifier(QualifierOperator.EARLIER_THAN, datetime.time(12, 0))))
        hierarchy.add_priority(Priority(PriorityType.ARRIVAL_BEFORE, 20, qualifier=Qualifier(QualifierOperator.EARLIER_THAN, datetime.time(18, 0))))
        hierarchy.add_priority(Priority(PriorityType.FEWER_TRANSFERS, 10))
        short_trip_user.hierarchy = hierarchy

        budget_minded_scoring = self.user.hierarchy.score(planner.find_trips())
        duration_minded_scoring = short_trip_user.hierarchy.score(planner.find_trips())

        self.assertNotEqual(budget_minded_scoring, duration_minded_scoring, "Trip scoring returns for the two different users should not be equal")
        print(
            "Budget-minded ranking: "
            + ", ".join(
                f"{trip.trip_id} ({score.score_value:.2f})"
                for score, trip in budget_minded_scoring
            ),
            flush=True,
        )
        print(
            "Duration-minded ranking: "
            + ", ".join(
                f"{trip.trip_id} ({score.score_value:.2f})"
                for score, trip in duration_minded_scoring
            ),
            flush=True,
        )

        first_trip_id_budget = budget_minded_scoring[0][1].trip_id
        first_trip_id_duration = duration_minded_scoring[0][1].trip_id

        self.assertNotEqual(first_trip_id_budget,first_trip_id_duration)
        print(
            f"Top trips differ: budget={first_trip_id_budget}, "
            f"duration={first_trip_id_duration}.",
            flush=True,
        )
```

Figure 4: Unit test code comparing trip rankings for budget-minded and duration-minded users.

The ranking system works in the scoring of deviation for the same trips when given the context of the different user's priority weighting. Not only is the first recommended trip different between the two; the relevance scores differ considering only the weight given to the duration and prices have changed.

8 Traveller Profile Comparisons

We'll compare three traveller profiles, with a different theme for each user.

- **Budget-focused traveller**: prioritises low price, departures after work at 18:00, and flexibility so if something changes they don't lose money.

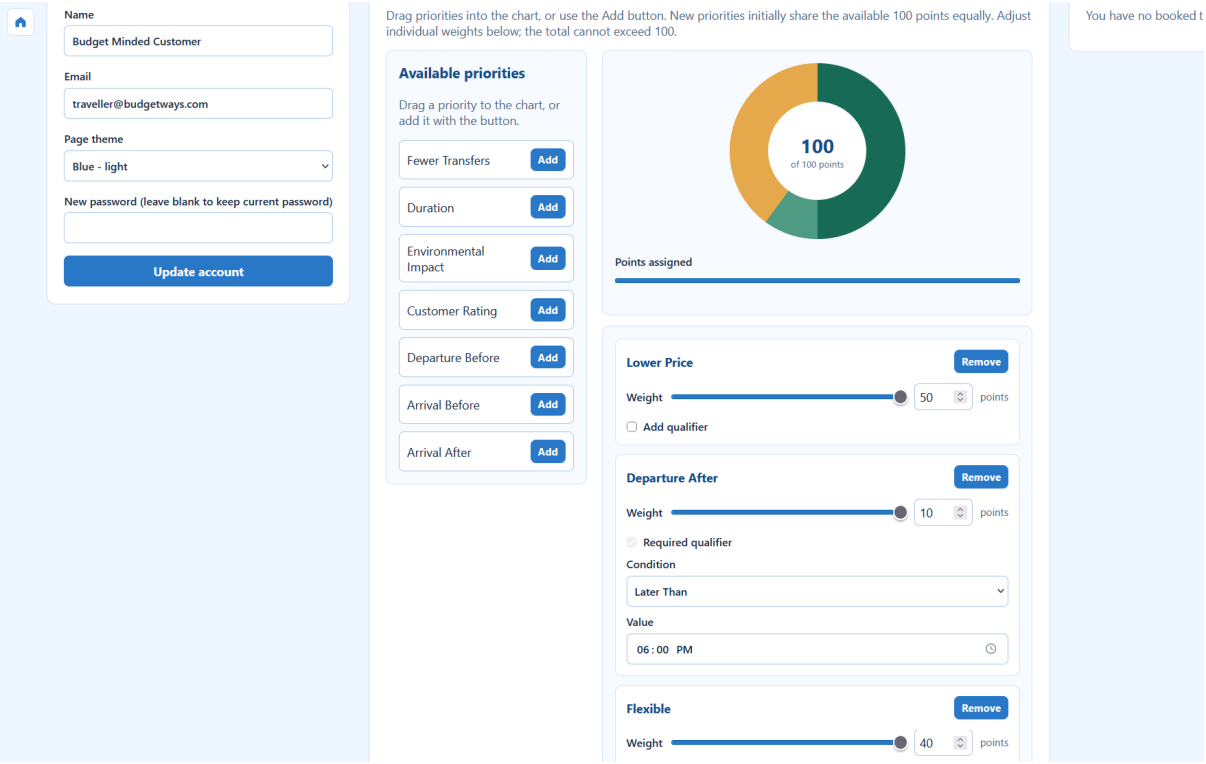


Figure 5: Priorities and weights of the Budget Minded user

- **Business traveller**: prioritises time efficiency, travel quality, and fewer transfers.

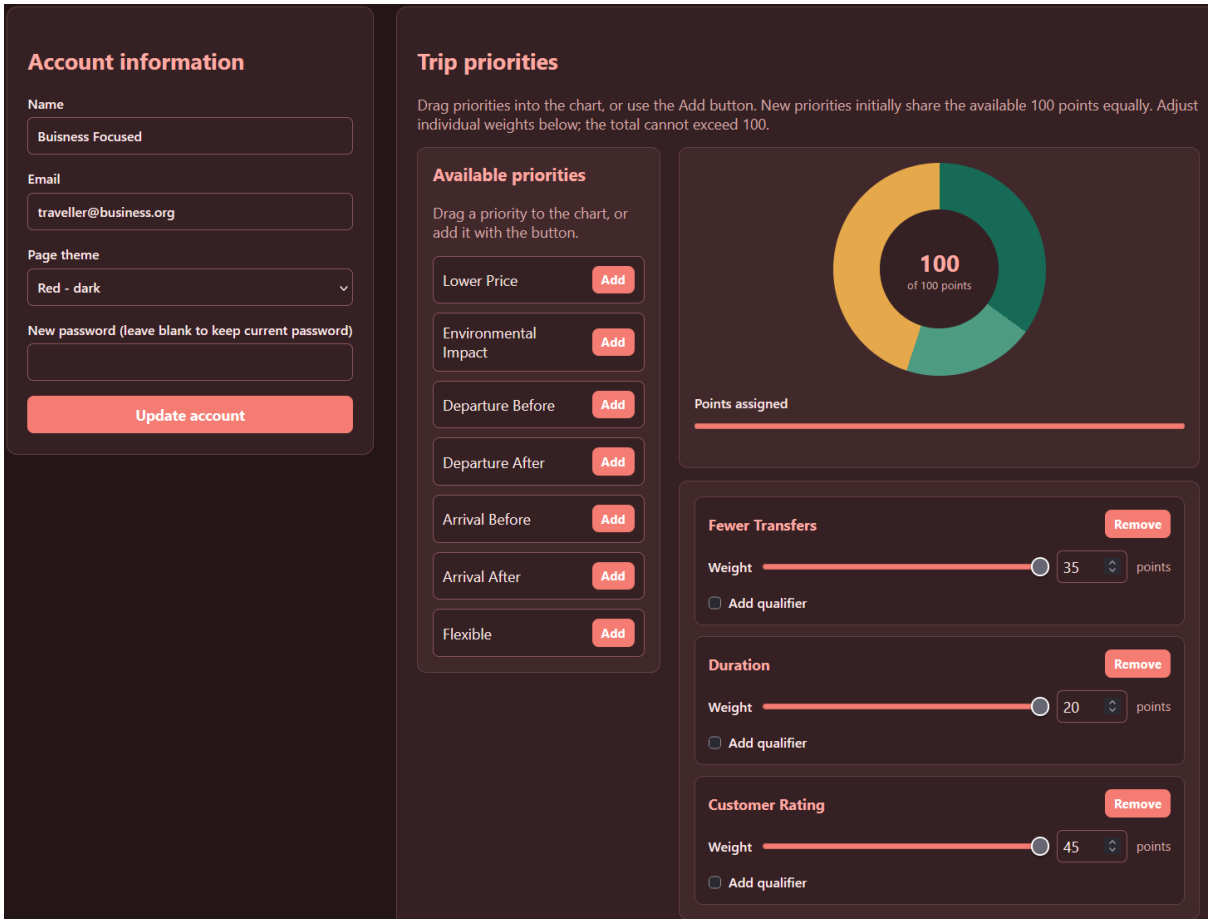


Figure 6: Priorities and weights of the Business user

- **Environmental-focused tourist**: prioritises lower Carbon impact, shorter duration, and fewer transfers.

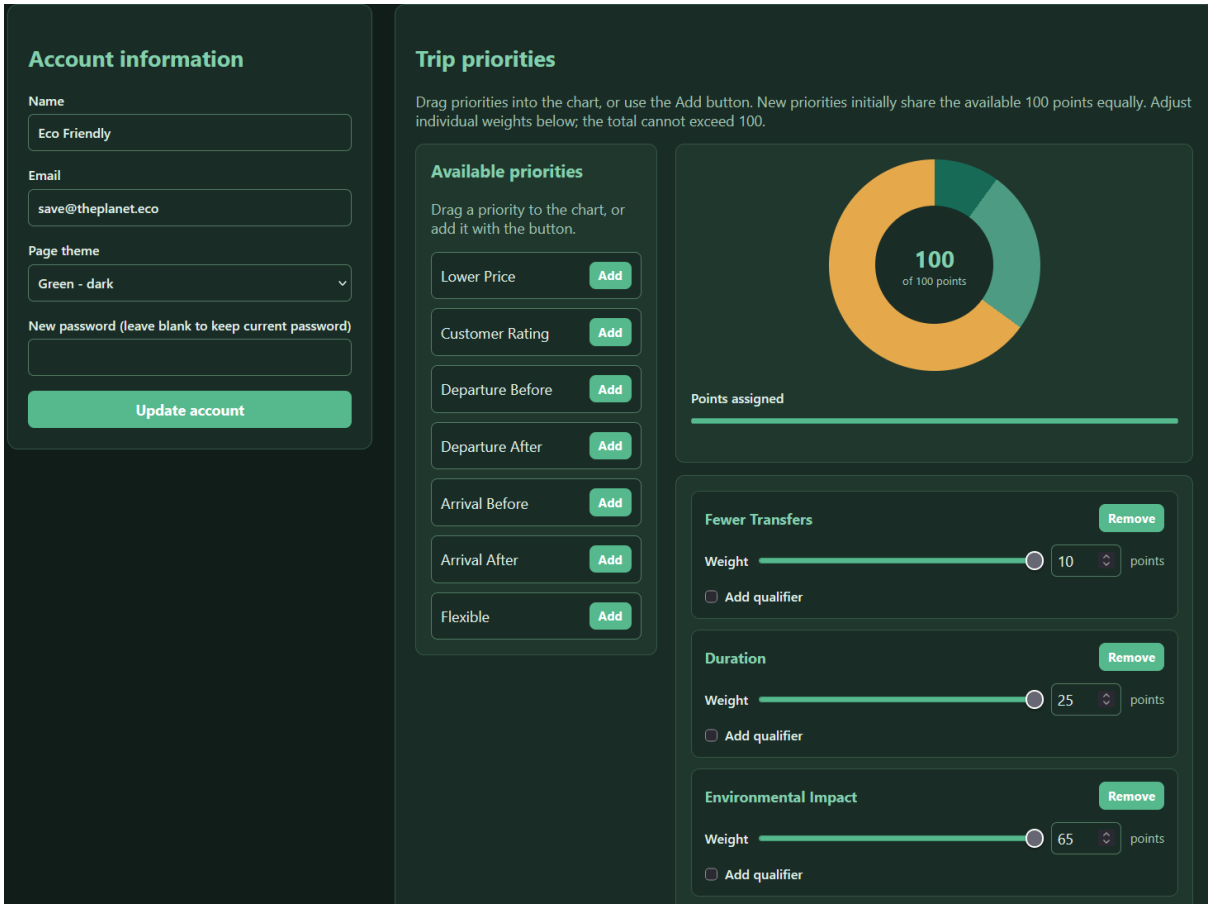
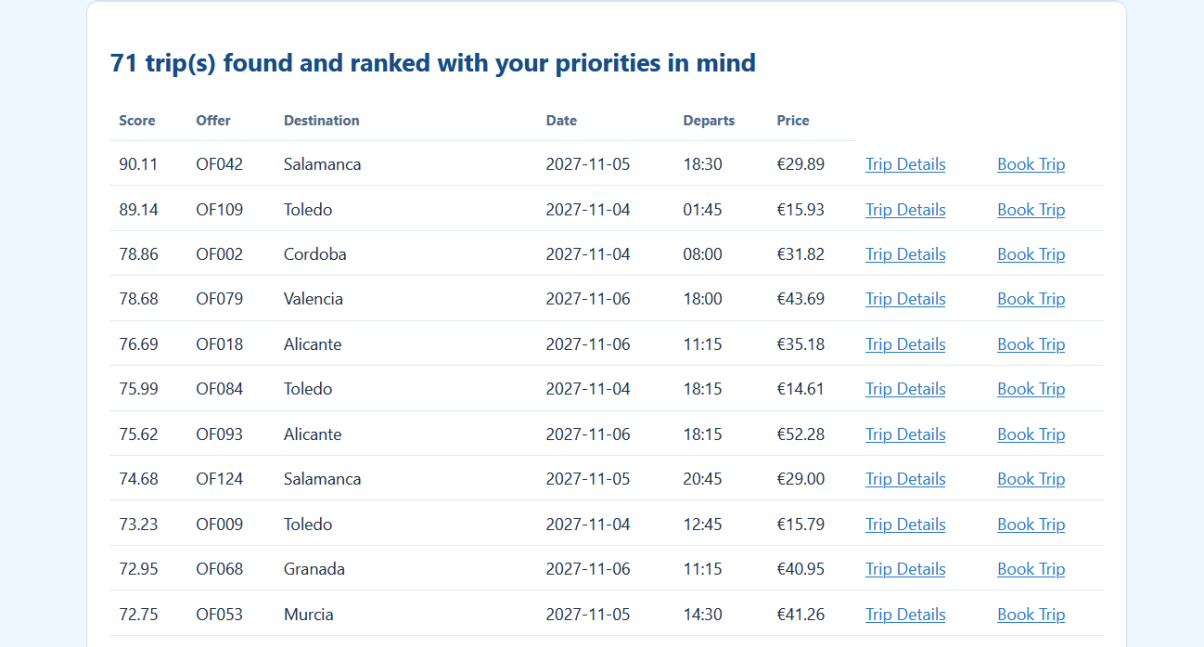


Figure 7: Priorities and weights of the Environmental minded user

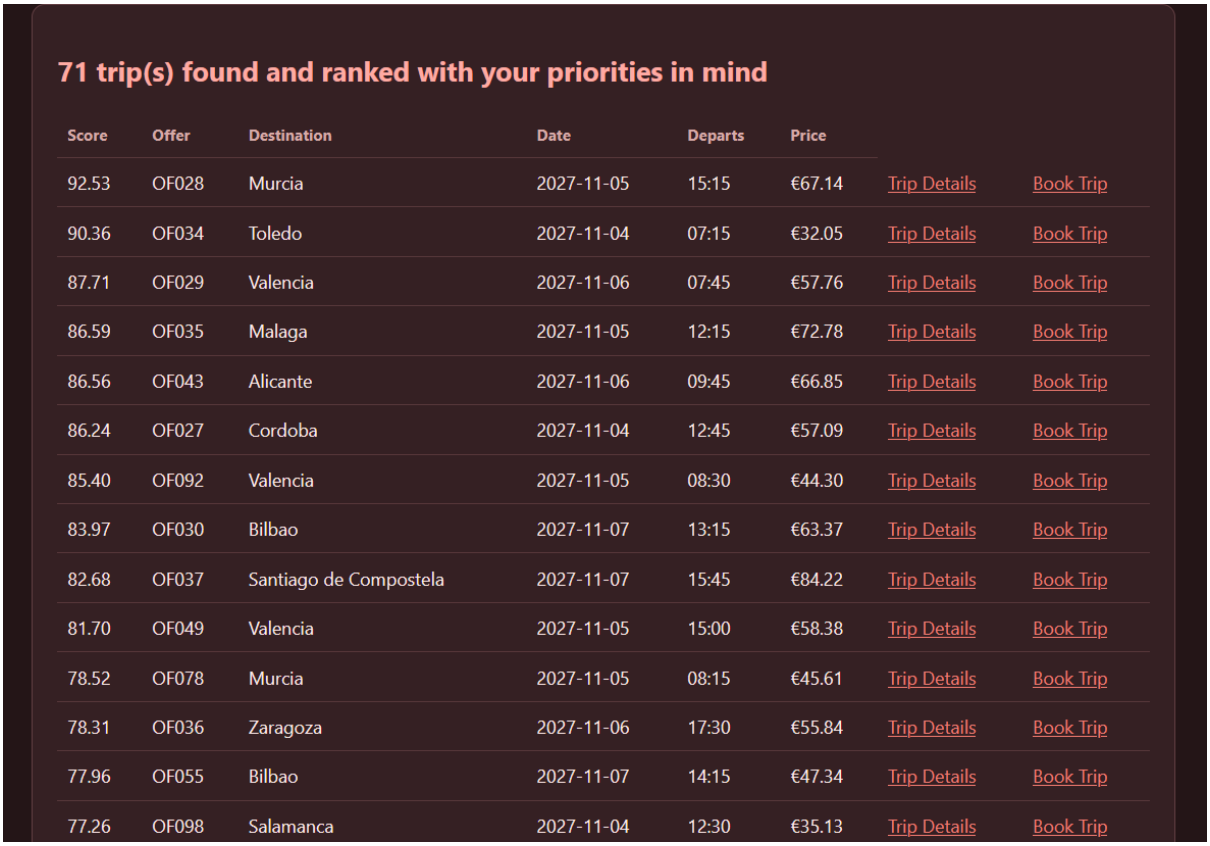
If we do a query against the dataset for all trips going anywhere (can be done by not selecting a destination or pressing control while you select destinations) in the time frame of 11-04-2027 to 11-07-2027 we can see our data set returns 71 trips for each user we have different ranked results as seen below:



71 trip(s) found and ranked with your priorities in mind

Score	Offer	Destination	Date	Departs	Price		
90.11	OF042	Salamanca	2027-11-05	18:30	€29.89	Trip Details	Book Trip
89.14	OF109	Toledo	2027-11-04	01:45	€15.93	Trip Details	Book Trip
78.86	OF002	Cordoba	2027-11-04	08:00	€31.82	Trip Details	Book Trip
78.68	OF079	Valencia	2027-11-06	18:00	€43.69	Trip Details	Book Trip
76.69	OF018	Alicante	2027-11-06	11:15	€35.18	Trip Details	Book Trip
75.99	OF084	Toledo	2027-11-04	18:15	€14.61	Trip Details	Book Trip
75.62	OF093	Alicante	2027-11-06	18:15	€52.28	Trip Details	Book Trip
74.68	OF124	Salamanca	2027-11-05	20:45	€29.00	Trip Details	Book Trip
73.23	OF009	Toledo	2027-11-04	12:45	€15.79	Trip Details	Book Trip
72.95	OF068	Granada	2027-11-06	11:15	€40.95	Trip Details	Book Trip
72.75	OF053	Murcia	2027-11-05	14:30	€41.26	Trip Details	Book Trip

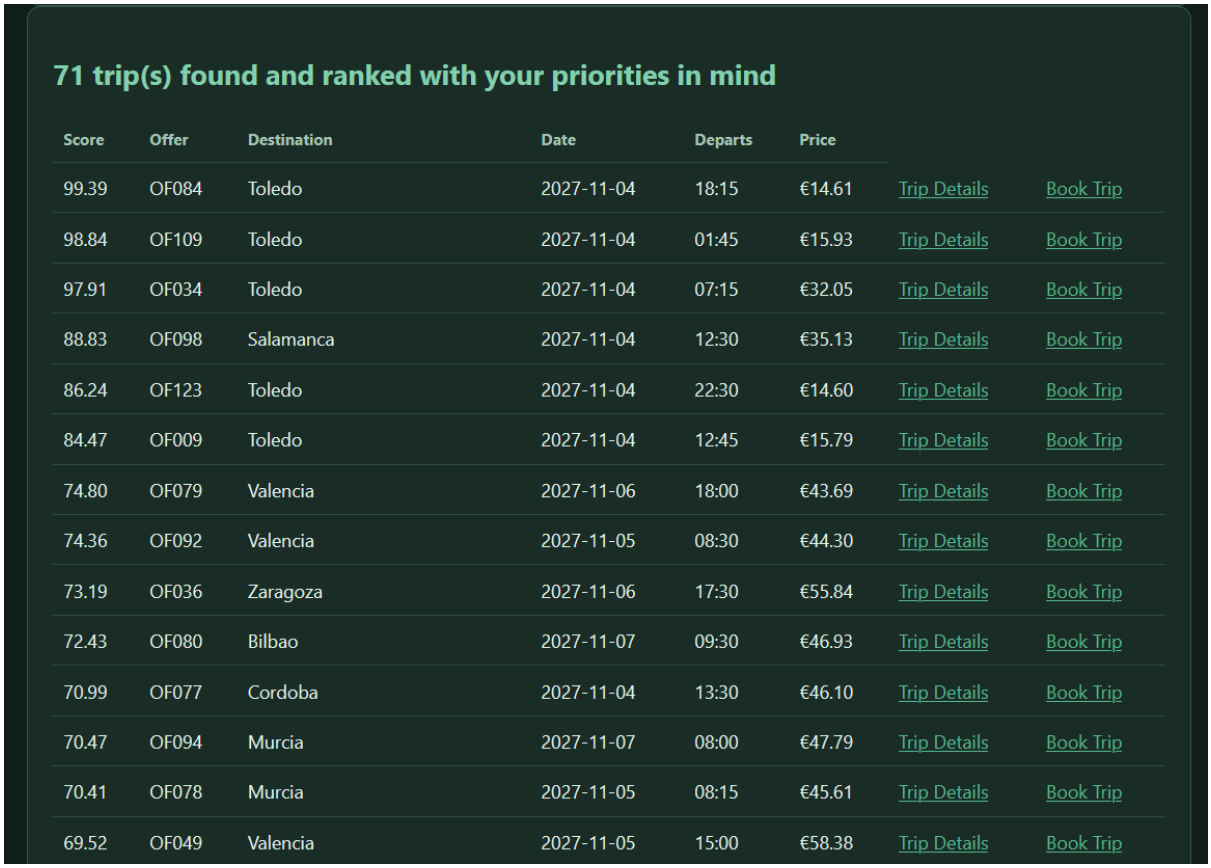
Figure 8: Top recommendations of the Budget Minded user



71 trip(s) found and ranked with your priorities in mind

Score	Offer	Destination	Date	Departs	Price		
92.53	OF028	Murcia	2027-11-05	15:15	€67.14	Trip Details	Book Trip
90.36	OF034	Toledo	2027-11-04	07:15	€32.05	Trip Details	Book Trip
87.71	OF029	Valencia	2027-11-06	07:45	€57.76	Trip Details	Book Trip
86.59	OF035	Malaga	2027-11-05	12:15	€72.78	Trip Details	Book Trip
86.56	OF043	Alicante	2027-11-06	09:45	€66.85	Trip Details	Book Trip
86.24	OF027	Cordoba	2027-11-04	12:45	€57.09	Trip Details	Book Trip
85.40	OF092	Valencia	2027-11-05	08:30	€44.30	Trip Details	Book Trip
83.97	OF030	Bilbao	2027-11-07	13:15	€63.37	Trip Details	Book Trip
82.68	OF037	Santiago de Compostela	2027-11-07	15:45	€84.22	Trip Details	Book Trip
81.70	OF049	Valencia	2027-11-05	15:00	€58.38	Trip Details	Book Trip
78.52	OF078	Murcia	2027-11-05	08:15	€45.61	Trip Details	Book Trip
78.31	OF036	Zaragoza	2027-11-06	17:30	€55.84	Trip Details	Book Trip
77.96	OF055	Bilbao	2027-11-07	14:15	€47.34	Trip Details	Book Trip
77.26	OF098	Salamanca	2027-11-04	12:30	€35.13	Trip Details	Book Trip

Figure 9: Top recommendations of the Business user



71 trip(s) found and ranked with your priorities in mind

Score	Offer	Destination	Date	Departs	Price		
99.39	OF084	Toledo	2027-11-04	18:15	€14.61	Trip Details	Book Trip
98.84	OF109	Toledo	2027-11-04	01:45	€15.93	Trip Details	Book Trip
97.91	OF034	Toledo	2027-11-04	07:15	€32.05	Trip Details	Book Trip
88.83	OF098	Salamanca	2027-11-04	12:30	€35.13	Trip Details	Book Trip
86.24	OF123	Toledo	2027-11-04	22:30	€14.60	Trip Details	Book Trip
84.47	OF009	Toledo	2027-11-04	12:45	€15.79	Trip Details	Book Trip
74.80	OF079	Valencia	2027-11-06	18:00	€43.69	Trip Details	Book Trip
74.36	OF092	Valencia	2027-11-05	08:30	€44.30	Trip Details	Book Trip
73.19	OF036	Zaragoza	2027-11-06	17:30	€55.84	Trip Details	Book Trip
72.43	OF080	Bilbao	2027-11-07	09:30	€46.93	Trip Details	Book Trip
70.99	OF077	Cordoba	2027-11-04	13:30	€46.10	Trip Details	Book Trip
70.47	OF094	Murcia	2027-11-07	08:00	€47.79	Trip Details	Book Trip
70.41	OF078	Murcia	2027-11-05	08:15	€45.61	Trip Details	Book Trip
69.52	OF049	Valencia	2027-11-05	15:00	€58.38	Trip Details	Book Trip

Figure 10: Top recommendations of the Environmentally focused user

Each user's recommendations have a unique order. If we look into each users highest choice we can also see a break down of the scoring as well as all other trip information.



Figure 11: Top trip of the Budget Minded user

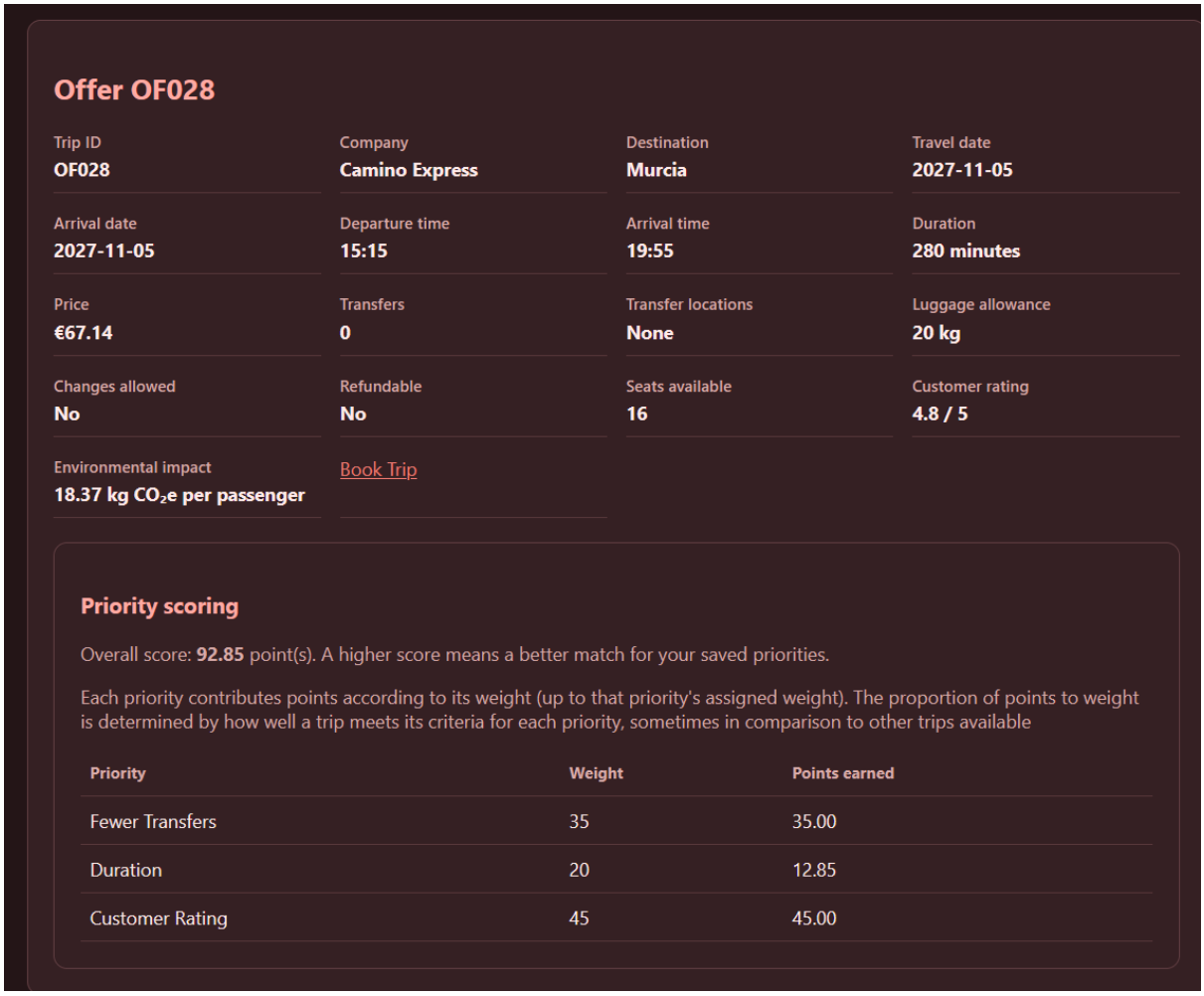


Figure 12: Top trip of the Business user



Figure 13: Top trip of the Environmentally focused user

We can see that each of the user's top recommended trips have a score breakdown on how well it meets up with their priorities, and they can tell from this alone how well it aligns with their preferences in comparison to other trips. For instance when a comparitive score like Environmental Impact, Price, or Customer Rating recieves all points of its share, the user knows that it's the lowest of the trips in the offering.

In the case of the budget users highest rated trip, the user can tell from this details pane that while it's close to being the cheapest the fact that it's flexible, and departs after the work day, and technically scores higher than the cheapest trip. in this example we see the value in the design desicion of adding this weighted scoring over allowing the user to sort columns in a table based on price alone.

9 Critical Evaluation and Limitations

As a basis for this kind of problem and use case, a web interface makes the most sense as it's an intuitive means for a user to browse trips and details and store information about them selves. It's a familiar format and as such requires little explanation on how to use the program. Secondly the decision to take certain priorities a user might have and allow them to be weighted allows for a more personalized service. As mentioned above, A user would be able to find appropriate trips on their own with a table that allows them to filter on things like Prices, Duration, customer rating, etc. on a scale of low to high or High to low. or a hard check for things like refundability. This approach however only takes one or two things into consideration. The weighted approach as demonstrates a more indepth application that can show a trip that aligns the most across all of the areas a user cares about. this applications design implementation also is modularized enough that future modifications would be trivial to add, like if a new administrator wanted to be able to add a new priority the framework exists in such a state that it'd only need to be updated in the priority objects code and then in the front end. Everything else that interacts with this data is generalized enough that it'll accept new data if structured in those two places.

This application is, however, limited in a few aspects. Because we're reading in data from a fixed CSV file, our breadth of trip complexity is limited to what's put into that file, and with it being fixed it'll eventually be unusable due to Dates being passed. The scoring also leaves its results as a floating value that can technically lead to ties, and it was intentionally decided that a floating point value of 2 decimal places would be sufficient scoring as the user should be able to look at two trips with the exact score and be able to determine on the extra information given by the details page what would be a tie breaker for themselves and accounting for that programmatically would weigh down the user experience and make possibly incorrect assumptions. In its current state, the established priority system may leave a priority that matters to a user unavailable. While this could be remedied by adding more and more priorities to the list, it was decided for this project that the given priorities would be sufficient also considering the user could manually query the data with anything else that data in a file may contain.

The scoring and normalization work, but some decisions don't fit the current data set. A prime example of this is that we currently score a priority of low number of transfers in a comparative evaluation methodology. In our dataset, trips only have 0 or 1 transfers which would work in a binary evaluation; however, it was a measured decision that it was possible for someone running the program to feed the application data with many transfers, and in that case a binary approach would be insufficient.

In deciding to also allow a user to set their own priorities we've also introduced a source of bias that while technically correct may come across as an insufficient solution. Assume a user has low carbon impact as a priority and low price as another and has weighed low carbon impact higher than price. They may see results returned for trips that all seem to have the similar impact but one being lower, while another with a significantly lower price then the rest of the trips that has an average impact. How would a user want these results ranked? In this implementation an outlier in lower prices may counter weigh the apparent carbon impact as it'll reduce the average pointing of trips because the price priority is getting smaller portions of points to the point where the second lower price is getting significantly less points then a person looking at it would decide to award it. A solution to this could be that every data point now has to look at the average distribution of values and adjust the normalization irrespective of outliers, but is that a better solution? It's possible some users would prefer outliers to have a heavier impact in what's returned. The problem has several solutions but there's no definitive correct path forward, and it's up to the applications design decisions to determine the best path forward; and in this case, I've decided that an outlier should cause a bit of trouble as that's probably

what the trip providers is probably hoping will happen.

10 AI Declaration of Use

The CSV file `data/trips.csv` that contains the data for our trips was populated using chat-GPT. It was given the context of the companies and the given locations. This felt acceptable because it allowed me to quickly generate 125 realistic trip offers and aside from deciding what kind of data would be present, coming up with valid seeming data at this scale seemed outside of the scope of the assignment. Generated data was reviewed, manually modified where necessary for example in the date and time fields when they were not in the right format for Python Date/Time object conversions.

I also utilized the coPilot AI built into VS code to help generate the templates of forms, style sheets, and javascript interactivity of the web pages used. Web development skills seemed out of scope for grading criteria, so this seemed appropriate. I did manually implement the Jinja scriptlet tags that allowed for a cleaner interaction between the flask interface to python, and the html based web pages.

Vs Code's auto code complete was also used to quickly build class and function declarations, and in the tripService.py initalizer function because it was only tedius attribute setting to parameter values but all other logic and design decisions made in the application were made without the aid of the AI tools present in the IDE

AI was used to comb through report to change some words like "it is", "can not", "is not" to their contraction equivalent and remove extra and trailing spaces to clean up report, but wording is all still and sentence structure is all still Human.